

Lab 8 – Solutions

November 18, 2024

This lab deals with polynomial and piecewise polynomial interpolation.

1 Polynomial interpolation

Given a function $f(x)$ the **interpolating function** in a finite dimensional space $V_k \subseteq V$ is

$$g(x) = \sum_{i=0}^n \alpha_i \phi_i(x) \quad \text{s.t.} \quad g(x_i) = f(x_i) \quad i = 0, 1, \dots, n$$

where $\{x_i\}_{i=0}^n$ and $\{\phi_i(x)\}_{i=0}^n$ are respectively sets of $k+1$ **nodes** and **basis functions**; the coefficients α_i are called **weights**.

Definition 8.1 (Lagrange interpolating polynomial). *Lagrange polynomials are defined taking as set of basis functions*

$$\varphi_j(x) = \prod_{j=0, i \neq j} \frac{(x - x_i)}{(x_j - x_i)}$$

and they are s.t. $\varphi_j(x_i) = \delta_{ij}$. The weights are $\alpha_i = f(x_i)$ and the Lagrange interpolating polynomial is

$$\Pi_n f(x) = \sum_{i=0}^k f(x_i) \varphi_i(x).$$

Theorem 8.1 (Interpolation error). *Let $\Pi_n f(x)$ be the interpolating polynomial of order n at the nodes $x_i \in [a, b]$. If $f(x) \in C^{n+1}([a, b])$ then*

$$E_n f(x) = f(x) - \Pi_n f(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i) \quad \xi \in [a, b], \quad \forall x \in [a, b].$$

If the nodes are equally spaced with step h , it holds

$$\|E_n f(x)\|_{L^\infty([a, b])} \leq \frac{\|f^{(n+1)}(x)\|_{L^\infty([a, b])}}{4(n+1)} h^{n+1}$$

This does not ensure a priori the uniform convergence of $\Pi_n f(x)$ to $f(x)$ for $n \rightarrow \infty$.

Exercise 8.1. Consider the following points

$$P_1(0, 5), P_2(1, 3), P_3(3, 5) \quad \text{and} \quad P_4(4, 12).$$

- Find the interpolation polynomial of minimum order that interpolates such points using Lagrange polynomials as basis functions.
- Plot the values attained by the obtained polynomial on a finer grid of nodes and verify that the polynomial actually interpolates the points.

c. Repeat the operation using MATLAB command `polyfit`.

Solution Exercise 8.1.

ex_8_1.m

```
%% Exercise 8.1
clc;clear;close all;
%% 1)
% Find the interpolation polynomial of minimum order that interpolates such points
% using Lagrange polynomials as basis functions.

x = [0 1 3 4]';
y = [5 3 5 12]';

% Since we have 4 points the interpolant polynomial of minimum order is of 4-1 = third order.
% Store the coefficients of the Lagrange polynomials in a matrix.
lagr_poly=zeros(numel(x), numel(x));
for j = 1:numel(x)
    % set difference: nodes now contains all the nodes except x(i).
    nodes = setdiff(x, x(j));
    % poly(r) returns the coefficients (in a vector) of the polynomial
    % whose root are the elements of r.
    % polyval(coefficients, hat_x) evaluates the polynomial with
    % given coefficients at the point hat_x.
    lagr_poly(:, j) = poly(nodes) / polyval(poly(nodes), x(j));
    % each column contains the coefficients of the j-th
    % basis function
end

%% 2)
% Plot the values attained by the obtained polynomial on a finer grid of nodes and verify that
% the polynomial actually interpolates the points.
xx = linspace(-0.5, 4.5, 1000);

% Display the basis functions

for i = 1:numel(x)
    name=sprintf("phi_%g",i);
    figure()
    hold on, box on
    axis([-1 5 -1.5 2.2])
    plot(xx, polyval(lagr_poly(:, i), xx), 'b-', 'LineWidth', 2)
    plot(x, polyval(lagr_poly(:,i), x), 'ro', 'LineWidth', 2, 'MarkerSize', 10)
    set(gca,'FontSize',16)
    set(gca,'LineWidth',1.5)
    xlabel('x','FontSize',16)
    ylabel('p_3(x)','FontSize',16)
    title(name)
end

% Sum the coefficients of the basis element with the weights:
coeff = lagr_poly * y;
yy = polyval(coeff, xx);
%
figure
hold on, box on
plot(x, y, 'ro', 'LineWidth', 2, 'MarkerSize', 10)
plot(xx, yy, 'b-', 'LineWidth', 2)
axis([-1 5 0 20])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('p_3(x)','FontSize',16)
title("Lagrange Interpolating Polynomial")
```

```

%% 3)
% Repeat the operation using MATLAB command polyfit
coeff2 = polyfit(x, y, 3)
[coeff, coeff2']

```

Exercise 8.2. Consider the problem of approximating the function $f(x) = \sin(x)$ in the interval $[0, \pi]$ with a polynomial of degree 9.

- Consider a set of equally spaced nodes x_i and find the interpolating polynomial using Lagrange basis polynomials.
- Find again the polynomial using `polyfit` command.
- Estimate the interpolation error using the related theorem. Then evaluate the interpolation polynomial in 1000 equally spaced points in the interval and plot the interpolation error. Motivate the result.
- Repeat the operations with polynomials of order $k = 1, \dots, 30$. Plot the behaviour of $\max_x |e_k(x)|$ as a function of k and comment the results.

Solution Exercise 8.2.

ex_8_2.m

```

%% Exercise 8.2
clc; clear; close all;
%% 1)
% Consider a set of equally spaced nodes  $x_i$  and find the interpolating
% polynomial using Lagrange basis polynomials.
% Since the polynomial has degree 9 we need 10 nodes.
f = @(x) sin(x);
x = linspace(0, pi, 10)';
y = f(x);
lagr_poly=zeros(numel(x), numel(x));
for i = 1:numel(x)
    nodes = setdiff(x, x(i));
    lagr_poly(:,i) = poly(nodes) / polyval(poly(nodes), x(i));
end
% Sum the coefficients of the basis element with the weights
coeff = lagr_poly * y;

%% 2)
% Find again the polynomial using polyfit command.

% polyfit
coeff2 = polyfit(x, y, 9);
% Check
[coeff, coeff2']

%% 3)
% Estimate the interpolation error using the related theorem.
% Then evaluate the interpolation polynomial in 1000 equally spaced points
% in the interval and plot the interpolation error. Motivate the result.
% We have to apply the formula
%  $e_k(x) \leq K_{k+1} h^{k+1} / (4(k+1))$ ,
% where the bound  $K_{k+1}$  on the absolute value of the  $k+1 = 10$ -th
% derivative is trivially obtained as  $K_{k+1} = 1$ . Thus ...
h=x(2)-x(1);

err_est = 1*(h)^10 / (4*10);
fprintf("Estimated Error: %d", err_est);
% Plot the obtained polynomial
xx = linspace(0, pi, 1000);
yy = f(xx);
pp = polyval(coeff, xx);

```

```

figure
% Just check that the polynomial interpolates the data-points
subplot(2,1,1)
plot(xx, yy, 'r-', 'LineWidth',2)
hold on, box on
plot(x, y, 'rx', 'LineWidth',2, 'MarkerSize',10)
axis([-0.1 pi+0.1 -0.1 1.1])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('p(x)','FontSize',16)
title("Lagrange Interpolating Polynomial")

err = abs(pp - yy);

subplot(2,1,2)
plot(xx, err, 'r-', 'LineWidth',2)
hold on, box on
plot(x, 0*x, 'rx', 'LineWidth',2, 'MarkerSize',10)
plot(x, err_est + 0*x, 'k--', 'LineWidth',2)
axis([-0.1 pi+0.1 -0.5e-7 7.5e-7])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('|e(x)|','FontSize',16)

% The interpolation on equally spaced nodes produces a non uniform error in the interval,
% which grows towards the extreme points.
%% 4)
% Repeat the operations with polynomials of order k =1,...,30.
% Plot the behaviour of abs(e_k(x)) as a function of k and
% comment the results.
degree=30;
errk=zeros(degree);
for k = 1:degree
    x = linspace(0, pi, k+1)';
    y = f(x);
    p = polyfit(x, y, k);

    xx = linspace(0, pi, 1000);
    yy = f(xx);
    pp = polyval(p, xx);
    errk(k) = max(abs(pp - yy)); % Take the maximum error
end

% Plot
figure("Name","Error as function of k")
semilogy(errk, 'b', 'LineWidth',2)
hold on, box on
axis([0 31 1e-14 5])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('max|e(x)|','FontSize',16)

% The error exponentially decreases with the number of nodes until k = 17.
% After that value of k the error increases.
% The warnings print out by MATLAB suggest that this is because the nodes
% become too close and the problem is ill-conditioned.

```

Exercise 8.3 (Runge's counterexample). Consider the function

$$f(x) = \frac{1}{1 + (5x)^2} \quad -1 \leq x \leq 1.$$

a. Verify that the interpolation polynomials $p_k(x)$ using equally spaced nodes are such that

$$\lim_{k \rightarrow \infty} |f(x) - p_k(x)| \neq 0.$$

- b. Overcome this problem using non uniformly distributed nodes, e.g. **Chebyshev nodes** which in $[-1, 1]$ are

$$x_i = \cos\left(\frac{1+2i}{2(k+1)}\pi\right) \quad i = 0, 1, \dots, k.$$

and can be mapped to a generic interval $[a, b]$ with the transformation

$$t_i = \frac{b-a}{2}x_i + \frac{a+b}{2}.$$

Solution Exercise 8.3.

ex_8_3.m

```
%% Exercise 8.4
clc;clear;close all;
%% 1)
% Verify that the interpolation polynomials  $p_k(x)$  using equally spaced
% nodes are such that
%  $\lim_{k \rightarrow \infty} \text{abs}(f(x) - p_k(x)) \sim 0$ 

f = @(x) 1./(1 + (5*x).^2);

xx = linspace(-1, 1, 1000);
fxx = f(xx);

figure("Name","Equally spaced nodes")
subplot(1,2,1)
hold on, box on
plot(xx, fxx, 'b-', 'LineWidth',2)
axis([-1.1 1.1 -0.4 1.2])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('f(x)','FontSize',16)

for k = 2:2:16
    x = linspace(-1, 1, k+1);
    fx = f(x);
    coef = polyfit(x, fx, k);
    yy = polyval(coef, xx);
    plot(xx, yy, 'r-', 'LineWidth',2)
    pause(1)
end

subplot(1,2,2)
%hold on, box on
%plot(xx, fxx, 'b-', 'LineWidth',2)
%axis([-1.1 1.1 -1.2 2.4])
%set(gca,'FontSize',16)
%set(gca,'LineWidth',1.5)
%xlabel('x','FontSize',16)
%ylabel('f(x)','FontSize',16)

%for k = 12:2:16
%    x = linspace(-1, 1, k);
%    fx = f(x);
%    coef = polyfit(x, fx, k-1);
%    yy = polyval(coef, xx);
%    plot(xx, yy, 'r-', 'LineWidth',2)
%    pause(1)
%end

% The interpolation polynomials approach the function in the middle
% of the interval, but close to the boundaries increasing oscillations appears.

%% 2)
```

```

% Overcome this problem using non uniformly distributed nodes, e.g. Chebyshev nodes
figure("Name", "Chebyshev nodes")
%subplot(1,2,1)
hold on, box on
plot(xx, fxx, 'b-', 'LineWidth',2)
axis([-1.1 1.1 -0.1 1.1])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('f(x)','FontSize',16)

for k = 2:2:20
    j = [0:k-1];
    xcheb = cos((2*j+1)/(2*(k))*pi);
    fcheb = f(xcheb);
    coef = polyfit(xcheb, fcheb, k-1);
    yy = polyval(coef, xx);
    plot(xx, yy, 'r-', 'LineWidth',2)
    pause(1)
end

%subplot(1,2,2)
%hold on, box on
%plot(xx, fxx, 'b-', 'LineWidth',2)
%axis([-1.1 1.1 -0.1 1.1])
%set(gca,'FontSize',16)
%set(gca,'LineWidth',1.5)
%xlabel('x','FontSize',16)
%ylabel('f(x)','FontSize',16)

%for k = [12:2:20]
% j = [0:k-1];
% xcheb = cos((2*j+1)/(2*(k))*pi);
% fcheb = f(xcheb);
% coef = polyfit(xcheb, fcheb, k-1);
% yy = polyval(coef, xx);
% plot(xx, yy, 'r-', 'LineWidth',2)
% pause(1)
%end

% The oscillating behavior is eliminated with Chebyshev nodes:
% the usage of this kind of nodes allow for a better distribution of the error,
% more uniformly in the interpolation interval. This is clear if we plot the
% monic polynomial, as an example for the case of 10 nodes.

%%
k = 10;
j = [0:k-1];
xeven = linspace(-1, 1, k);
xcheb = cos((2*j+1)/(2*k)*pi);

coef_even = poly(xeven);
coef_cheb = poly(xcheb);

pi_even = polyval(coef_even, xx);
pi_cheb = polyval(coef_cheb, xx);

figure
plot(xx, pi_even, 'r', 'LineWidth',2)
hold on, box on
plot(xeven, 0*xeven, 'rx', 'LineWidth',2, 'Markersize',10)
plot(xx, pi_cheb, 'b', 'LineWidth',2)
plot(xcheb, 0*xcheb, 'bx', 'LineWidth',2, 'Markersize',10)
axis([-1.1 1.1 -13e-3 3e-3])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('pi(x)','FontSize',16)

% The monic polynomial obtained with equally spaced nodes shows high peaks

```

% next to the extremes of the interval. Instead the one related to the
 % Chebyshev nodes has a series of maxs and mins of the same height.

Notice that the commands `polyfit` and `polyval` can be used to compute both the interpolation and a least-square approximation

Note

`polyfit(xnodes, ynodes, N)`

$N + 1 = \#x_nodes \implies$ Lagrange interpolation;
 $N + 1 \neq \#x_nodes \implies$ least square interpolation.

2 Piecewise polynomial interpolation

When using n equally spaced nodes, uniform convergence of the interpolating polynomial is not guaranteed for $n \rightarrow \infty$.

Definition 8.2 (Piecewise linear interpolation). *Given of nodes: $x_0 < x_1 < \dots < x_n$, we denote by I_i the interval $[x_i, x_{i+1}]$ and by H the maximum length of these intervals. We denote by $\Pi_1^H f$ the piecewise linear interpolating polynomial of f given by:*

$$\Pi_1^H f(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}(x - x_i) \quad \forall x \in I_i.$$

Theorem 8.2 (Interpolation error). *If $f \in C^2(I)$, where $I = [x_0, x_n]$, then*

$$\|f(x) - \Pi_1^H f(x)\|_{L^\infty(I)} \leq \frac{H^2}{8} \|f''(x)\|_{L^\infty(I)}$$

This bound allows us to ensure the uniform convergence of the interpolating polynomial to f .

Exercise 8.4. a. Consider $f(x) = \sin(x)$ in the interval $[0, \pi]$: compute the interpolation with $n = 1, 2, 4, 8, 16, 32$ elements, find the error in infinity norm and verify it converges quadratically with the distance between two nodes h .

b. Consider now the Runge function

$$f(x) = \frac{1}{1 + (5x)^2} \quad -1 \leq x \leq 1 :$$

compute the interpolation with $n = 1, 2, 4, 8, 16, 32$ elements, find the error in infinity norm and verify it converges quadratically with the distance between two nodes h .

c. Interpolate the Runge function using a piecewise cubic spline (`spline` command) with $n = 1, 2, 4, 8, 16, 32$ subintervals. Plot the result.

Solution Exercise 8.4.

ex_8_4.m

```
%% Exercise 8.4
clc;clear;close all;
%% 1)
% Consider (x)=sin(x) in the interval [0, pi]: compute the interpolation
% with n = 1,2,4,8,16,32 elements, find the error in infinity norm
% and verify it converges quadratically with the distance between two nodes h.
a = 0; b = pi; f = @(x) sin(x);
x_plot = linspace(a, b, 101);

n_vect = [1 2 4 8 16 32];

figure
```

```

for i = 1:numel(n_vect)
    n = n_vect(i);

    x = linspace(a, b, n+1);
    y = f(x);

    y_plot = interp1(x, y, x_plot);

    plot(x_plot, f(x_plot), 'b-', x_plot, y_plot, 'r-', x, y, 'rx', 'LineWidth', 2, 'MarkerSize'
        , 8)
    axis([a-0.2 b+0.2 -0.1+min(f(x_plot)) 0.1+max(f(x_plot))])
    set(gca, 'FontSize', 16)
    set(gca, 'LineWidth', 1.5)

    error(i) = max(abs(f(x_plot) - y_plot));
    pause(1.5)
end

figure
loglog(n_vect, error, 'bx-', 'LineWidth', 2, 'MarkerSize', 8)
hold on, box on
loglog(n_vect, 1./n_vect.^2, 'g-', 'LineWidth', 2)
axis([0.7 100 1e-4 3])
set(gca, 'FontSize', 16)
set(gca, 'LineWidth', 1.5)
xlabel('n', 'FontSize', 16)
ylabel('error', 'FontSize', 16)
legend('error', 'O(h^2)', 'Location', 'ne');

% From the graph we can conclude that the convergence order is 2,
% as predicted by the theory.
p = -diff(log(error)) / log(2)

%% 2)
% Consider now the Runge function
% f(x) = 1/(1+(5x)^2) :
% compute the interpolation with n = 1,2,4,8,16,32 elements,
% find the error in infinity norm and verify it converges quadratically with
% the distance between two nodes h.
clear;
a = -1; b = 1;
f = @(x) 1./(1+25*x.^2);
x_plot = linspace(a, b, 101);

n_vect = [1 2 4 8 16 32 64 128];

figure
for i = 1:numel(n_vect)
    n = n_vect(i);

    x = linspace(a, b, n+1);
    y = f(x);

    y_plot = interp1(x, y, x_plot);
    plot(x_plot, f(x_plot), 'b-', x_plot, y_plot, 'r-', x, y, 'rx', 'LineWidth', 2, 'MarkerSize'
        , 8)
    axis([a-0.2 b+0.2 -0.1+min(f(x_plot)) 0.1+max(f(x_plot))])
    set(gca, 'FontSize', 16)
    set(gca, 'LineWidth', 1.5)

    error(i) = max(abs(f(x_plot) - y_plot));
    pause(1.5)
end

figure
loglog(n_vect, error, 'bx-', 'LineWidth', 2, 'MarkerSize', 8)
hold on, box on
loglog(n_vect, 1./n_vect.^2, 'g-', 'LineWidth', 2)
axis([0.7 600 1e-4 3])
set(gca, 'FontSize', 16)

```

```

set(gca,'LineWidth',1.5)
xlabel('n','FontSize',16)
ylabel('error','FontSize',16)
legend('error','O(h^2)','Location','ne');

% From the graph we can conclude that the convergence order is 2, as predicted by the theory.

p = -diff(log(error)) / log(2)
%% 3)
figure
for i = 1:numel(n_vect)
    n = n_vect(i)

    x = linspace(a, b, n+1);
    y = f(x);

    y_plot = spline(x, y, x_plot);
    plot(x_plot, f(x_plot), 'b-', x_plot, y_plot, 'r-', x, y, 'rx', 'LineWidth', 2, 'MarkerSize'
        , 8)
    axis([a-0.2 b+0.2 -0.1+min(f(x_plot)) 0.1+max(f(x_plot))])
    set(gca,'FontSize', 16)
    set(gca,'LineWidth', 1.5)
    error(i) = max(abs(f(x_plot) - y_plot));
    pause(1.5)
end

```

Exercise 8.5. Consider the function $f(x) = e^x$ in the interval $[-1, 1]$ and the problem of finding a piecewise linear interpolating polynomial on a set of nodes $\{x_i\}$.

- Determine how many equally spaced nodes x_i are needed in order to get an interpolation error with infinity norm smaller than 10^{-3} .
- Construct the interpolating polynomial using such number of nodes, compute the error numerically and check if the request is satisfied.

Solution Exercise 8.5.

ex_8_5.m

```

%% Exercise 8.5
clc; clear; close all;
%% 1)
% Determine how many equally spaced nodes x_i are needed in order
% to get an interpolation error with infinity norm smaller than 10^-3.

f = @(x) exp(x);
% We know that (in particular, C = 1/8)
% norm(err, inf) <= h^2/8 norm(f''(x), inf)
% <= h^2/8 norm(e^x, inf)
% <= h^2/8 e < \eps = 10^-3
% thus
h = sqrt( 1e-3 * 8 / exp(1) )

% and the required number of nodes is
n = ceil(2 / h) + 1 % = 38
%% 2)
% Construct the interpolating polynomial using such number
% of nodes, compute the error numerically and check if
% the request is satisfied.
x = linspace(-1, 1, n);
y = exp(x);

x_plot = linspace(-1, 1, 1000);

y_plot = interp1(x,y,x_plot);

figure

```

```

plot(x_plot, f(x_plot), 'b-', x_plot, y_plot, 'r-', x, y, 'rx', 'LineWidth', 2, 'MarkerSize',
8)
axis([-1.1 1.1 0 3])
set(gca,'FontSize', 16)
set(gca,'LineWidth', 1.5)

err = max(abs(f(x_plot) - y_plot))

% The requirement is satisfied.

```

Exercise 8.6. Consider the function

$$f(x) = \left| x - \frac{\pi}{12} \right| \quad -1 \leq x \leq 1.$$

- Verify that the interpolating polynomials $\Pi_n f(x)$ based on equally spaced nodes exhibit the Runge's phenomenon.
- Compute the piecewise linear interpolation (`interp1` command) based on $n = 1, 2, 4, 8, 16, 32$ intervals and compute the associated error with the respect to infinity norm. Is the quadratic convergence ensured in such a case?
- Interpolate the function using a piecewise cubic spline (`spline` command) with $n = 1, 2, 4, 8, 16, 32$ subintervals. Plot the result.
- Interpolate the function using the least square approach bu setting the number of nodes $n = 10$ and varying the degree of the interpolant. Plot the results and check when the least-square approximation exactly matches the values `y_nodes` in correspondence with the `x_nodes`.

Solution Exercise 8.6.

ex_8_6.m

```

%% Exercise 8.6
clc; clear; close all
%% 1)
% Setting the function and its visualization
a = -1; b = 1;
f = @(x) abs(x - pi/12);

xx = linspace(a, b, 1000);
fxx = f(xx);

figure("Name","Runge's phenomenon")
% matrix of plots: one row, two columns, first subplot
hold on, box on
plot(xx, fxx, 'b-', 'LineWidth',2)
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('x','FontSize',16)
ylabel('f(x)','FontSize',16)

% a) The interpolating polynomials n f (x ) using equally
% spaced nodes show the Runges phenomenon

for k = 2:2:16
    x = linspace(a, b, k+1);
    fx = f(x);
    coef = polyfit(x, fx, k);
    yy = polyval(coef, xx);
    plot(xx, yy, 'r-', 'LineWidth',1)
    axis([a-0.2 b+0.2 -0.1+min(f(xx)) 0.1+max(f(xx))])
    pause(1.5)
end

% The interpolation polynomials approach the function in the middle of the
% interval, but close to the boundaries increasing oscillations appears.

```

```

%% 2)
% Computation of the piecewise linear interpolation (interp1 command)
% and calculation of the convergence order ( f(x) is not C2(I) function!!)

% clear all
a = -1; b = 1;
f = @(x) abs(x - pi/12);
x_plot = linspace(a, b, 1000);

% Number of subintervals: 2^k
% n_vect = [1 2 4 8 16 32];
n_vect = [1 2 4 8 16 32 64 128 256];

figure
for i = 1:numel(n_vect)
    n = n_vect(i)
    x = linspace(a, b, n+1);
    y = f(x);
    y_plot = interp1(x, y, x_plot);
    plot(x_plot, f(x_plot), 'k-', x_plot, y_plot, 'r-', x, y, 'ro', 'LineWidth', 2, 'MarkerSize'
        , 8)
    axis([a-0.2 b+0.2 -0.1+min(f(x_plot)) 0.1+max(f(x_plot))])
    set(gca,'FontSize', 16)
    set(gca,'LineWidth', 1.5)

    %error(i) = max(abs(f(x_plot) - y_plot));
    error(i) = norm((f(x_plot) - y_plot),'inf');
    %pause
end

% determination of the element width
H = (b-a)./n_vect;

% Are 32 subintervals enough to grafically assess
% the convergence order?

figure
% the dash after the color and the markersize denotes
% points linked by lines
loglog(H, error, 'b*-', 'LineWidth', 2, 'MarkerSize', 8)
hold on, box on
% linear order
loglog(H, H, 'k-', 'LineWidth', 2)
hold on, box on
% quadratic order
loglog(H, H.^2, 'r-', 'LineWidth', 2)
%axis([0.7 600 1e-4 3])
set(gca,'FontSize',16)
set(gca,'LineWidth',1.5)
xlabel('step size: h=(b-a)/n','FontSize',16)
ylabel('error','FontSize',16)
legend('error', 'O(h)', 'O(h^2)', 'Location', 'ne');

% A second order convergence cannot be expected, since the function is not smooth enough.
% Calculation of the convergence order
p = -diff(log(error)) / log(2)

%% c) Interpolate the function using a piecewise cubic spline (spline
% command) with n = 1, 2, 4, 8, 16, 32 elements.
n_vect = [1 2 4 8 16 32];
figure
for (i = 1:numel(n_vect))
    n = n_vect(i)
    x = linspace(a, b, n+1);
    y = f(x);
    y_plot = spline(x, y, x_plot);
    plot(x_plot, f(x_plot), 'b-', x_plot, y_plot, 'r-', x, y, 'ro', 'LineWidth', 2, 'MarkerSize'
        , 8)
    axis([a-0.2 b+0.2 -0.1+min(f(x_plot)) 0.1+max(f(x_plot))])

```

```

    set(gca,'FontSize', 16)
    set(gca,'LineWidth', 1.5)
    pause(1.5)
end

%% d) Interpolation of the points generated with the above function with nodes  $n = 10$ 
% and using the least square approach with varying the degree of the interpolant.

% close all

a = -1; b = 1;
f = @(x) abs(x - pi/12);
x_plot = linspace(a, b, 1000);
x_nodes = linspace(a, b, 10);
y_nodes = f(x_nodes);

figure
plot(x_nodes, y_nodes, 'rO');
hold on;

for ii = 4:8
    v = polyfit(x_nodes, y_nodes, ii);
    y_plot = polyval(v, x_plot);
    plot(x_plot, y_plot)
    pause
end

figure
plot(x_nodes, y_nodes, 'rO');
hold on;
v = polyfit(x_nodes, y_nodes, 9);
y_plot = polyval(v, x_plot);
plot(x_plot, y_plot)

```